TECHNICAL DOCUMENTATION

Uptime Monitor

A production website-monitoring service with email alerting and a public status page — designed, built and deployed end to end on AWS.

Author: Ana Mukherjee · Software Developer, FeedSense AI
Stack: NestJS · PostgreSQL · React · Docker · AWS (EC2, SQS, SES, IAM, CloudWatch, CloudFront)
Repository: github.com/Arnabi13/uptime-monitor
Document date: 27 October 2026 · **Region:** AWS ap-southeast-2 (Sydney)

Contents

1. Overview & goals
2. Technology stack
3. System architecture
   - Component diagram
   - The request & check lifecycle
4. Data model
   - Entities
   - Indexing strategy
5. Application design
   - Backend modules
   - The checking engine
   - Security design
6. API reference
7. AWS services in detail
   - Account & region context
   - EC2
   - SQS
   - SES
   - IAM
   - CloudWatch
   - CloudFront
8. Deployment
   - Containers & images
   - Docker Compose & Nginx
   - The deploy script
   - Database migrations
9. CI/CD pipeline
10. Operations runbook
11. Cost & free-tier notes
12. Roadmap & what's left
13. Glossary & interview talking points

1 · Overview & goals

Uptime Monitor watches public websites, records their response times, raises an incident and emails the owner when a site goes down, and publishes a shareable public status page.

The project was built as a portfolio piece to demonstrate full-stack engineering together with real, end-to-end cloud deployment — not a toy app running only on `localhost`. Every layer, from the database schema to the HTTPS edge, is deployed and operating on AWS.

What it does

- **Monitoring.** A user registers a URL with a check interval (60 s–24 h), an HTTP method, a timeout and an optional expected status code. A background engine checks it on schedule.
- **Incidents & alerting.** After a configurable number of consecutive failures (default 3), an incident opens and a "down" email is sent. On the next success the incident resolves and a "recovery" email is sent.
- **Dashboards.** A React single-page app shows per-monitor uptime (24 h / 7 d / 30 d), response-time charts, the last 30 checks, and incident history.
- **Public status page.** Each account has a shareable page (a random 12-character slug) listing only the monitors the owner marked public, with names, status and daily uptime — and nothing else.

Design goals

Production-grade	Migrations (never schema auto-sync), constraints enforced in the database, graceful shutdown, health checks, rate limiting, non-root containers.
Secure by default	Monitors are private; the server defends against SSRF and DNS-rebinding because it makes HTTP requests to user-supplied URLs.
Cloud-native	A real queue decouples scheduling from checking; alerting uses a managed email service; credentials come from an instance role, not static keys.
Cost-free	Runs entirely within the AWS Free Tier: one `t3.micro`, SQS, SES and CloudWatch free allowances.

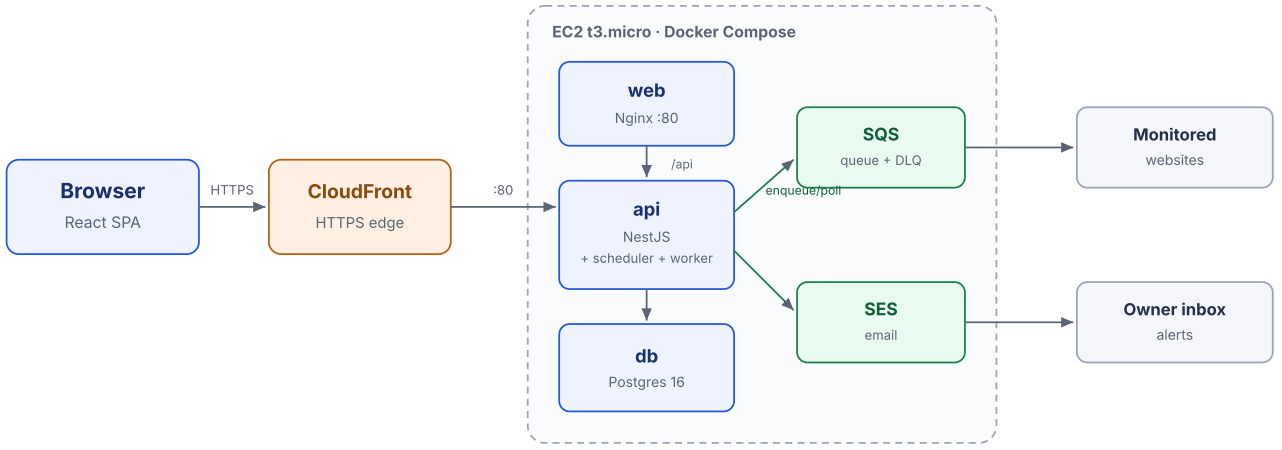
2 · Technology stack

Layer	Choice	Why
API	NestJS 11 (TypeScript)	Structured modules, dependency injection, class-validator for DTOs.
Database	PostgreSQL 16 via TypeORM	Strong constraints, row locking, `date_bin` aggregation, BRIN indexes.
Auth	JWT (HS256) + bcrypt	Stateless tokens; password hashing capped at bcrypt's 72-byte limit.
Checking engine	AWS SQS (standard queue + DLQ)	Decouples the scheduler from slow/hanging HTTP checks; at-least-once retry.
Alerting	AWS SES v2	Deliverable transactional email without running a mail server.
Frontend	React 18 + Vite + TanStack Query	Server-state caching, polling, instant cache updates; hand-rolled SVG charts.
Containers	Docker + Docker Compose	Identical stack locally and on EC2; multi-stage builds.
Edge / web	Nginx + CloudFront	One origin for app + API; CloudFront adds free HTTPS.
CI/CD	GitHub Actions	Tests on every push; auto-deploy to EC2 on merge to `main`.
Monitoring	CloudWatch Logs + Alarms → SNS	Centralised container logs; instance health alarm by email.

One origin, everywhere. The frontend only ever calls `/api/...` (never an absolute API URL). In dev, Vite proxies `/api` to Nest; in production, Nginx proxies it to the API container; at the edge, CloudFront forwards everything to the one origin. This means no CORS preflight on every request and no API URL baked into the build.

3 · System architecture

3.1 · Component diagram



Blue = app containers on one EC2 instance. **Green** = managed AWS services reached via the instance's IAM role. Only Nginx (:80) is exposed; Postgres and the API are reachable only on the private Compose network.

3.2 · The request & check lifecycle

Two independent flows run in the system. A **user request** (left) and a **background check** (right):

1 · Scheduler tick
Every minute, a cron job finds active monitors whose `next_check_at` has passed.

2 · Claim
It advances `next_check_at` immediately inside a locked transaction, so no check is enqueued twice.

3 · Enqueue
One SQS message per due monitor (`{ monitorId }`).

4 · Worker poll
The worker long-polls SQS and receives the job.

5 · Re-validate
DNS is re-resolved and every resolved IP re-checked as public (defeats DNS rebinding).

6 · HTTP check
The request is pinned to the validated IP; timing and status recorded.

7 · Persist
One `checks` row + status update, in one transaction.

8 · Alert
Crossing the failure threshold opens an incident and sends an SES email; a later success resolves it.

Why a queue sits in the middle. Deciding what is due is fast and predictable; making an HTTP request to someone else's server is slow and can hang. Putting SQS between them means a slow site never blocks the next scheduling tick, the worker processes at its own pace, and if the worker restarts, jobs wait in the queue instead of being lost. SQS's at-least-once delivery is safe here because a check is idempotent — running it twice just checks the URL twice.

4 · Data model

4.1 · Entities

Table	Purpose	Key columns
`users`	Accounts	`email` (unique), `password_hash` (never selected by default)
`monitors`	One row per watched URL	`url`, `host`, `interval_seconds`, `timeout_ms`, `expected_status_code`, `status`, `next_check_at`, `consecutive_failures`, `is_public`
`checks`	One row per check run (chart/uptime source)	`bigint id`, `monitor_id`, `checked_at`, `is_up`, `response_time_ms`, `status_code`, `cause`
`incidents`	One row per down period	`monitor_id`, `cause`, `opened_at`, `resolved_at` (NULL = open)
`status_pages`	Public page settings	`slug` (random 12-char), `title`

CONSTRAINTS LIVE IN THE DATABASE, NOT JUST THE CODE

- `UNIQUE (user_id, url)` — a user can't add the same URL twice. The app catches the unique-violation and returns **409** instead of checking first (which would race).
- `CHECK interval_seconds >= 60` and `CHECK timeout_ms BETWEEN 1000 AND 30000`.
- `ON DELETE CASCADE` from monitors to checks/incidents, so deleting a monitor cleans up its history.

4.2 · Indexing strategy

Index	Serves	Note
`idx_monitors_due (is_active, next_check_at)`	Scheduler's "which monitors are due?" query, every minute	Cheap even with thousands of monitors.
`uq_monitors_user_url (user_id, url)`	Uniqueness *and* lookups by user	No separate `user_id` index needed — this one's leftmost column is `user_id`.
`idx_checks_monitor_time (monitor_id, checked_at)` — B-tree	"Last 30 checks" and "checks since T"	A LATERAL join walks it backwards, stopping after 30 rows per monitor.
`checked_at` — **BRIN**	The daily retention delete (`checked_at < X`)	Rows arrive in time order, so BRIN stores min/max per block range — a few KB where a B-tree would be MBs.
`idx_incidents_monitor_open (monitor_id, resolved_at)`	"Is there an open incident for this monitor?"	Enforces at most one open incident per monitor in practice.

> **Aggregate in SQL, ship ~90 numbers.** The response-time chart asks for a range; Postgres buckets it with `date_bin` (15-min / 2-hour / 8-hour slots) and returns ~90 rows with `count(*) FILTER (WHERE is_up)` and `avg()`. 30 days of a 60-second monitor is 43,200 rows — the browser receives 90.

5 · Application design

5.1 · Backend modules

Module	Responsibility
`auth`	Register, login, JWT strategy & guard. Same error/timing for unknown email vs. wrong password (no account enumeration). JWT pinned to HS256. Stricter rate limit on auth routes.
`monitors`	CRUD, ownership checks, abuse limits, stats & history queries.
`checks`	Check-history entity (one row per check).
`incidents`	Open/resolve logic; one open incident per monitor.
`scheduler`	Cron: claims due monitors, enqueues SQS jobs; daily check-history pruning.
`worker`	SQS consumer: HTTP check (+ DNS pinning), updates monitor/incidents, triggers alerts.
`queue`	Thin SQS client wrapper (send / receive / delete).
`notifications`	SES email alerts; never throws — a flaky mail provider must not stop checks.
`status-page`	Owner settings + the unauthenticated public endpoint.
`common/url-safety`	The SSRF guard (unit-tested).
`health`	`/api/health` — app + DB, used by Docker and Nginx.
`config`	Env validation — the app refuses to start on bad config.

5.2 · The checking engine

CLAIMING DUE MONITORS SAFELY

```
// scheduler.service.ts — runs every minute
SELECT ... FROM monitors
 WHERE is_active AND next_check_at <= now()
 ORDER BY next_check_at ASC
 LIMIT 50
 FOR UPDATE SKIP LOCKED;     // <- makes multi-instance safe
// then, in the same transaction, advance next_check_at for each claimed row
```

`SKIP LOCKED` is what makes it safe to run more than one scheduler later without two of them enqueueing the same monitor in the same tick. Advancing `next_check_at` *before* the check runs means a slow cycle can't double-book a monitor.

ONE ALERT PER DOWN PERIOD

A monitor opens an incident (and emails) only after crossing `ALERT_FAILURE_THRESHOLD` (default 3) **consecutive** failures — one slow response isn't an outage. Once open, further failures update status but don't open a second incident or send a second email. The incident resolves (and a recovery email goes out) on the next success. This is enforced by always checking for an existing open incident (`resolved_at IS NULL`) before creating one.

CHECK & STATUS IN ONE TRANSACTION

Every check writes a `checks` row *and* updates the monitor's status inside the same transaction, so the chart and the status badge can never disagree.

5.3 · Security design

Risk	Mitigation
SSRF — users type URLs the server will fetch (e.g. `http://169.254.169.254`, the EC2 metadata service that hands out IAM credentials)	Every URL's hostname is resolved and accepted only if *every* resolved IP is public unicast. Internal names (`localhost`, `*.internal`, single-label names) are rejected before DNS.
DNS rebinding — a domain points to a public IP until accepted, then repoints to a private one	The worker re-validates right before each request and connects directly to the IP it just validated (custom DNS `lookup`), while still sending the real `Host` header and TLS SNI. Validation and connection look at the same address.
Abuse / hammering a third-party site	Minimum 60-second interval, max monitors per account, max monitors per host.
Create-limit race	Creating a monitor locks the user's row (`SELECT … FOR UPDATE`) so concurrent creates run one at a time. (The e2e suite fires 8 concurrent creates against a limit of 5 and asserts exactly 5 succeed.)
Ownership disclosure	Asking for another user's monitor returns **404**, not 403 — the API never confirms someone else's IDs exist.
Public-page leakage	The public response exposes names, status and daily uptime only — no URLs, no failure causes (which can reveal internals like `ECONNREFUSED`), no emails, no IDs. The slug is random and rotatable.
XSS stealing the JWT (stored in `localStorage`)	React escapes all output; the app never uses `dangerouslySetInnerHTML`. Any 401 on a request that sent a token logs the user out and clears the cache.

6 · API reference

All routes are prefixed with `/api`. Monitor routes require `Authorization: Bearer <token>`.

Method	Route	Description
POST	`/auth/register`	Create account → `{ accessToken, user }`
POST	`/auth/login`	Log in → `{ accessToken, user }`
GET	`/auth/me`	Current user
GET	`/monitors`	List your monitors
POST	`/monitors`	Create a monitor
GET	`/monitors/:id`	Get one monitor
PATCH	`/monitors/:id`	Edit, pause (`isActive:false`) or resume. Only changed fields are sent, so renaming doesn't re-run DNS validation.
DELETE	`/monitors/:id`	Delete
GET	`/monitors/overview`	Uptime 24 h, avg response & last 30 checks for every monitor, in one call (avoids N+1 over HTTP)
GET	`/monitors/:id/stats`	Uptime 24 h/7 d/30 d, avg response, open incident
GET	`/monitors/:id/history?range=…`	Response time + up/down in ~90 buckets (`24h|7d|30d`)
GET	`/monitors/:id/incidents`	Last 20 incidents
GET	`/status-page`	Your status-page slug, title, public-monitor count
PATCH	`/status-page`	Rename it
POST	`/status-page/rotate`	New random slug; the old one 404s
GET	`/public/status/:slug`	**No auth.** Public status-page data (`Cache-Control: public, max-age=30`)
GET	`/health`	App + database health (Docker & Nginx use it)

CREATE-MONITOR FIELDS

`name`, `url` (required); `method` (`HEAD` \| `GET`, default HEAD); `intervalSeconds` (60–86400, default 300); `timeoutMs` (1000–30000, default 10000); `expectedStatusCode` (default: any 2xx/3xx); `isPublic` (default false).

7 · AWS services in detail

7.1 · Account & region context

Account	`142852456807` — a member account under an AWS Organization (custom-branded console, not a direct aws.amazon.com signup)
Region	**ap-southeast-2 (Sydney)** — the only region allowed by the organization's Service Control Policy (SCP). Resource creation in any other region is denied.
Billing	Free plan with credit; the whole project stays within Free-Tier allowances.

> **A real-world lesson.** An early `sqs:CreateQueue` AccessDenied wasn't a blanket SQS block — it was the org SCP restricting resource creation to `ap-southeast-2`, while the setup script was hardcoded to `ap-south-1`. Fix: point everything at Sydney. This is exactly the kind of guardrail real AWS Organizations impose.

7.2 · EC2 — the compute

Instance	`i-0fbc9ec71ef64d223` (name: uptime-monitor), **t3.micro**, Amazon Linux 2023
Address	`3.26.46.254` · DNS `ec2-3-26-46-254.ap-southeast-2.compute.amazonaws.com`
Security group	`sg-0aa8a690c61a19d25` (launch-wizard-1): inbound **80** from `0.0.0.0/0`, **22** from your IP; outbound all
IAM role	`uptime-monitor-ec2-role` — attached so the app gets short-lived credentials automatically (no access keys on disk)

The instance runs Docker and the whole stack via Compose. It is sized for 1 GB RAM: Postgres is tuned with `shared_buffers=64MB` and `max_connections=30` (the API's TypeORM pool uses 10). Docker's JSON logs are capped (`max-size 10m`, `max-file 3`) and old image layers are pruned on each deploy, because the root disk is small.

7.3 · SQS — the work queue

Main queue	`uptime-monitor-checks` (standard)
Dead-letter queue	`uptime-monitor-checks-dlq`, wired via `RedrivePolicy` with `maxReceiveCount = 3`

The scheduler sends one message per due monitor. The worker long-polls, processes, and deletes the message on success. On failure it does *not* delete — the message becomes visible again and is retried; after 3 receives SQS moves it to the DLQ for inspection instead of retrying forever. A malformed message (not JSON) is logged and deleted immediately so it can't poison the queue.

7.4 · SES — transactional email

Verified sender	`arnabimukherjee2001@gmail.com` (Verified, Sydney)
API	SES v2 `SendEmail`
Config switch	`ALERT_EMAIL_FROM` — blank disables email (alerts are logged only)

> **The SES sandbox.** New accounts are sandboxed: SES will only send *to* addresses that are also verified. For this project, verifying one address covers both sender and recipient. Alerts route to the email of the account that owns the monitor, so testing uses an app account registered with that verified address. Mail sent "via amazonses.com" from a Gmail sender often lands in spam because Gmail can't sign it — the production fix is a custom domain with DKIM/SPF/DMARC records.

7.5 · IAM — least-privilege access

Two identities, each with the minimum it needs:

Identity	Used by	Permissions
`uptime-monitor-backend` (IAM user, access key)	Local development	SQS send/receive/delete on the one queue
`uptime-monitor-ec2-role` (instance role)	The EC2 instance in production	SQS on the queue; `ses:SendEmail`; CloudWatch Logs (`CreateLogGroup/Stream`, `PutLogEvents`) — via inline policy `uptime-monitor-extras`

Using an **instance role** in production means no long-lived access keys sit on the server; the SDK fetches short-lived, auto-rotating credentials from the instance metadata service. (Which is exactly why the SSRF guard blocks `169.254.169.254`.)

7.6 · CloudWatch — logs & alarms CONFIGURED

An opt-in Compose override (`docker-compose.cloudwatch.yml`) switches all containers to the `awslogs` log driver when `CLOUDWATCH_LOGS=true` is set in the environment. Logs flow into log group `uptime-monitor` (streams `api`, `web`, `db`), with retention set to 7 days so storage stays free. A planned alarm on the instance's `StatusCheckFailed` metric notifies an SNS topic by email if the box becomes unreachable.

7.7 · CloudFront — the HTTPS edge IN PROGRESS

CloudFront sits in front of the instance to provide free HTTPS (via the default `*.cloudfront.net` certificate, so no custom domain is required). It is configured to:

- Use the EC2 **public DNS** as the origin, HTTP only on port 80 (CloudFront → origin stays HTTP; the viewer gets HTTPS).
- Redirect HTTP → HTTPS for viewers.
- Allow all HTTP methods and forward everything (CachingDisabled + AllViewer), so login POSTs and the `Authorization` header pass through.

With CloudFront in front of Nginx there are **two** proxy hops, which is why `TRUST_PROXY=2` (see §8).

8 · Deployment

8.1 · Containers & images

Three containers, one `docker-compose.prod.yml`:

Service	Image	Notes
`db`	`postgres:16-alpine`	Volume-backed; tuned for 1 GB RAM; `pg_isready` healthcheck.
`api`	built from `backend/Dockerfile`	Multi-stage build — the TypeScript compiler and dev deps never ship. Runs as non-root `node`, has a healthcheck, and `exec`s node so it receives `SIGTERM` for graceful shutdown.
`web`	built from `frontend/Dockerfile`	Multi-stage: Vite builds the static app, then it's served by Nginx. The only service that publishes a port (**80**).

Postgres and the API are reachable only on the private Compose network, so the security group only needs ports 80 and 22.

8.2 · Docker Compose & Nginx

Nginx serves the built React app and proxies `/api/` to the API container. Static assets (hashed filenames) are cached forever; `index.html` is never cached, so a new deploy is picked up on the next page load. Unknown paths fall back to `index.html` for React Router. Security headers (`X-Content-Type-Options`, `X-Frame-Options: DENY`, `Referrer-Policy`) are set, and the Nginx version is hidden.

```
Internet ──:80──▶ web (Nginx: React + /api proxy) ──▶ api (NestJS) ──▶ db (Postgres)
                                                          └──▶ SQS / SES (via instance role)
```

TRUST_PROXY — A SUBTLE BUT IMPORTANT SETTING

Behind proxies, every request appears to come from the proxy's IP. Rate limiting needs the *real* client IP, read from `X-Forwarded-For`. The app trusts exactly as many proxy hops as really exist:

`TRUST_PROXY=0`	Local dev (no proxy)
`TRUST_PROXY=1`	Nginx only
`TRUST_PROXY=2`	CloudFront → Nginx

> **Why it must match reality.** Trusting *more* hops than exist would let a client spoof its IP with a forged `X-Forwarded-For` header and dodge rate limits. So the number is set to the true hop count, not "on/off".

8.3 · The deploy script

`deploy/deploy.sh`, run from the repo root on the server:

```
sh deploy/deploy.sh
  # 1. requires backend/.env (fails fast if missing)
  # 2. docker compose --env-file backend/.env -f docker-compose.prod.yml up -d --build
  #    (+ docker-compose.cloudwatch.yml when CLOUDWATCH_LOGS=true)
  # 3. docker image prune -f   (reclaim disk)
  # 4. docker compose ps        (show status)
```

The database password is defined once, in `backend/.env`, and passed through with `--env-file` so Compose can create the database from the same values.

8.4 · Database migrations

`synchronize` is always off — the schema only ever changes through reviewed migration files. The container's entrypoint runs `typeorm migration:run` before starting the app; if a migration fails, the container exits rather than starting against the wrong schema (`set -e`). Migrations: InitialSchema, Incidents, ChecksAndStatusPage.

9 · CI/CD pipeline

Two GitHub Actions workflows. Tests gate every change; deploys happen only after tests pass on `main`.

ci.yml — on every PR and push to main

Job	Steps
`backend`	Spins up a real **Postgres 16** service container, runs `npm ci`, build, unit tests, and e2e tests — which run the *actual migrations* against that database, so schema bugs surface in CI.
`frontend`	`npm ci`, typecheck, tests, build.
`docker`	Builds both images to catch Dockerfile breakage before it reaches the server.

deploy.yml — after CI succeeds on main

1. Triggered by a successful CI run on `main` (or manually via "Run workflow").
2. SSHes to EC2 using a **dedicated deploy key** (separate from the personal `.pem`, so it can be revoked on its own).
3. Checks out the *exact commit CI tested* (`git checkout --force $SHA`), not whatever `main` is by then, and runs `deploy/deploy.sh`.
4. Polls `/api/health` and fails the run if the app doesn't come back healthy.

GitHub secret	Value
`EC2_HOST`	The instance's public IP
`EC2_SSH_KEY`	The deploy key's private half (PEM text)
`EC2_USER` / `EC2_APP_DIR`	Optional overrides (default `ec2-user` / `~/uptime-monitor`)

> **Trade-off.** GitHub's runners have no fixed IP, so automatic deploys require SSH (port 22) reachable from anywhere. Key-only login (password auth is off) keeps that acceptable for a portfolio project; a hardened setup would use a self-hosted runner or a bastion/VPN instead.

10 · Operations runbook

Connect to the server

```
ssh -i ~/.ssh/uptime-monitor-key.pem ec2-user@3.26.46.254
```

If it refuses or times out, your home IP changed — update the SSH rule's source in the security group to "My IP". The app (port 80) stays reachable regardless.

Redeploy by hand

```
cd ~/uptime-monitor && git pull && sh deploy/deploy.sh
```

Change configuration

```
nano ~/uptime-monitor/backend/.env   # edit a value
sh deploy/deploy.sh                   # apply
```

Inspect / troubleshoot

App logs	`docker logs uptime-monitor-api-1 --since 15m`
Container status	`docker compose -f docker-compose.prod.yml ps`
Health	`curl http://3.26.46.254/api/health`
DLQ depth	SQS console → `uptime-monitor-checks-dlq` → Messages available

Common situations

Symptom	Likely cause / fix
Can't SSH (`Connection refused/timeout`)	Home IP changed → update SG rule, or port 22 not open to the runner for deploys.
Alerts not arriving	Recipient not SES-verified (sandbox), or `ALERT_EMAIL_FROM` blank, or in spam.
Deploy fails at SSH step	SG doesn't allow GitHub runners (set 22 to `0.0.0.0/0`), or `EC2_SSH_KEY` secret malformed.
Messages piling in the DLQ	A check keeps throwing — inspect one message and the API logs.
Instance address changed after stop/start	Attach an Elastic IP (see roadmap) to keep it stable.

11 · Cost & free-tier notes

Service	Free allowance	This project's usage
EC2 t3.micro	750 hours/month (12 months)	One always-on instance ≈ 730 h/month
SQS	1 million requests/month	A handful of monitors = thousands/month
SES	62,000 emails/month (sent from EC2)	A few alert emails
CloudWatch Logs	5 GB ingestion + storage	Capped container logs, 7-day retention
CloudFront	1 TB data out / 10M requests (always-free)	Negligible
Data transfer out	100 GB/month (always-free)	Negligible

Everything is sized to stay at or near zero cost. The main things to watch are leaving CloudWatch log retention unset (logs kept forever) and running more than one always-on instance.

12 · Roadmap & what's left

Item	Status	Notes
Backend API, checking engine, frontend	DONE	Deployed and operating.
EC2 + Docker Compose + Nginx	DONE	Single-instance production stack.
SQS checking pipeline + DLQ	DONE	Scheduler → queue → worker.
SES email alerting	DONE	Verified; down + recovery emails confirmed.
CI/CD (GitHub Actions)	DONE	Tests on every push; auto-deploy on `main`.
CloudWatch logging + alarm	PENDING	Logging wired; set retention, create SNS topic + `StatusCheckFailed` alarm.
CloudFront HTTPS	PENDING	Create distribution; set `CORS_ORIGIN` + `TRUST_PROXY=2`; redeploy.
Elastic IP	OPTIONAL	Keep the server's address stable across stop/start.
Postgres backup to S3	OPTIONAL	Nightly `pg_dump` → S3; data currently lives in one container volume.
Custom domain + DKIM/SPF/DMARC	OPTIONAL	Gets alert emails out of spam and off the SES sandbox.

13 · Glossary & interview talking points

Glossary

SSRF	Server-Side Request Forgery — tricking a server into making requests to internal addresses it shouldn't reach.
DNS rebinding	Changing a domain's DNS answer after it's been validated, to slip past an IP check.
DLQ	Dead-letter queue — where SQS parks messages that fail too many times, instead of retrying forever.
Instance role	An IAM role attached to EC2 so code gets short-lived AWS credentials without storing keys.
SCP	Service Control Policy — an org-wide guardrail (here, "only Sydney").
BRIN	Block Range Index — a tiny index for naturally-ordered columns like timestamps.
SKIP LOCKED	A Postgres clause that lets concurrent workers claim different rows without blocking each other.

Talking points

- **"Why a queue?"** Scheduling is fast and predictable; checking a third-party site is slow and can hang. SQS decouples them, survives worker restarts, and gives safe at-least-once retries with a DLQ backstop.
- **"How is it secure against SSRF?"** Resolve the hostname, accept only if every IP is public, re-validate right before each request, and connect pinned to the validated IP — so DNS can't change between the check and the connection.
- **"How do you deploy?"** Push to `main` → GitHub Actions runs unit + e2e tests against a real Postgres, then SSHes to EC2, checks out the tested commit, redeploys with Compose, and verifies `/api/health`.
- **"How do credentials work on the server?"** No static keys — an EC2 instance role hands the SDK short-lived, auto-rotating credentials, scoped to just SQS, SES and CloudWatch Logs.
- **"What did you learn about real AWS?"** Org SCPs that lock you to one region, the SES sandbox, Free-Tier boundaries, and why `trust proxy` must equal the true number of hops.